

GRUPO I – Programação em Prolog (6.5 val.)

1.1) Suponha a seguinte representação do conhecimento relativo a objectos

objecto(mesa1, vidro).	pesos(mesa1, 10).	material(vidro, fragil).	sobre(cadeira, mesa1).
objecto(cadeira, madeira).	pesos(cadeira, 2).	material(madeira, forte).	sobre(mesa2, mesa1). %mesa2 em cima
objecto(mesa2, madeira).	pesos(mesa2, 3).	material(ferro, forte).	sobre(bigorna, mesa2).
objecto(bigorna, ferro).	pesos(bigorna, 20).		sobre(jarra, bigorna).
objecto(jarra, vidro).	pesos(jarra, 0.5).		sobre(caixa, jarra). % caixa em cima
objecto(caixa, madeira).	pesos(caixa, 2.5).		

1.1 a) Escreva o predicado **sobreTipo(+Tipo,-Lista)** que retorne uma lista (com o formato indicado abaixo) com todos os objectos que estão colocados directamente sobre objectos de um dado tipo. (1.5 Val.)

Exemplo:

?- sobreTipo(madeira, L).

L = [bigorna*sobre*mesa2]

?- sobreTipo(vidro, L).

L = [cadeira*sobre*mesa1, mesa2*sobre*mesa1, caixa*sobre*jarra]

1.1 b) Escreva o predicado **partem(-ListaObjectos)**, que retorne a lista de objectos que vão partir dado o conhecimento apresentado. Um objecto pode partir se for feito de um material frágil e se estiver debaixo de um conjunto de objectos que tenham um peso que seja mais do dobro do peso desse objecto. Sugestão implemente uma função auxiliar **peso_total(+Obj, -PesoTot)**, que calcule o peso total suportado por um dado objecto. (2 Val.).

Exemplo:

?- partem(L).

L = [mesa1-2.8., jarra-5] %Partem a mesa pois tem 2.8 vezes o seu peso em cima (2+3+20+0.5+2.5=28kg) e a jarra que tem 5 vezes o seu peso em cima (pesa 0.5kg e tem a caixa com 2.5kg em cima).

1.2) Considere as seguintes possíveis definições do predicado **membro(X, Lista)**:

membro1(X, [X|Tail]).

membro2(X, [X|Tail]) :- !.

membro1(X, [Y|Tail]) :- membro1(X, Tail).

membro2(X, [Y|Tail]) :- membro2(X, Tail).

Ambas as definições são, de facto úteis dependendo da aplicação. Descreva a alto-nível: Em que medida as definições diferem? Quando se deve preferir uma à outra? Quais são as soluções possíveis dos objectivos **membro1(X, [1,2,3,4])** e **membro2(X, [1,2,3,4])**. (1 Val.)

1.3) Quais as respostas do Prolog às seguintes questões? Justifique. (2 Val.).

1.3 a) ?- N is N+1.

1.3 b) ?- N == 5, N2 is N+1.

1.3 c) ?- N = 5, N is N+1.

1.3 d) ?- functor(Term, f, 5), arg(4, Term, 2).

1.3 e) ?- [A|B] =.. [C|D].

GRUPO II – Processamento de Listas (7 val.)

2.1) Suponha as seguintes implementações do predicado **junta3listas(+L1, +L2, +L3, -LR)**, que concatena três listas. Diga justificando e exemplificando qual das duas versões é a mais eficiente. (1 Val.).

junta3listas1(L1, L2, L3, LR):-

append(L1, L2, LR1),

append(LR1, L3, LR).

junta3listas2(L1, L2, L3, LR):-

append(L2, L3, LR1),

append(L1, LR1, LR).

append([], L, L).

append([H|R1], L, [H|R2]):- append(R1, L, R2).

2.2) Defina em Prolog o predicado **separa_posit_neg(+Lista, -ListaNeg, -ListaPos)** que separa uma lista de inteiros arbitrários em duas listas, contendo a primeira os números negativos e a segunda os números positivos. (1.5 Val.).

Exemplos:

?- separa_posit_neg([-1, 2, 0, 5, -5], LN, LP).

LN = [-1, -5], LP = [2, 5].

?- separa_posit_neg([1, 5, 2, 3, 0, -2, -3, 6], LN, LP).

LN = [-2, -3], LP = [1, 5, 2, 3, 6].

2.3) Uma mãe, um pai, 2 filhas, 2 filhos, um polícia e um ladrão querem atravessar o rio. Dispõem de um barco (onde só cabem duas pessoas). O barco só pode ser conduzido pela mãe, pelo pai ou pelo polícia. A Mãe não pode ficar sem a ajuda do pai numa margem com qualquer dos filhos. O pai não pode ficar sem a ajuda da mãe numa margem com qualquer das filhas. O ladrão não pode ficar sozinho com qualquer elemento da família sem a presença do polícia. Considera-se que sempre que o barco atinge uma margem todos os passageiros saem do mesmo (mesmo que depois voltem a entrar).

**Pai Mae Filho Filho Filha
Filha Ladrão Policia**

B
a
r
c
o

Suponha que os estados inicial e final do problema são representados pelos seguintes predicados contendo listas ordenadas alfabeticamente:

estado_inic([barco, filha, filha, filho, filho, ladrao, mae, pai, policia], []).

estado_final([], [barco, filha, filha, filho, filho, ladrao, mae, pai, policia]).

2.3 a) Defina um predicado **operadores(-Oper, +Margem1, +Margem2)** que dadas as listas de elementos que estão nas margens 1 e 2, retorne um a um, todos os operadores com formato **Dir – ListaViajantes** possíveis numa dada situação. **Dir** poderá ser **baixo** ou **cima**. Considere os operadores possíveis mesmo que a sua aplicação deixe uma das margens numa situação inválida (1 Val.).

Exemplos:

?- operadores(Oper, [barco, filha, filho, ladrao, policia], [filha, filho, mae, pai]).

Oper = baixo – [policia] ; Oper = baixo – [filha, policia] ; Oper = baixo – [filho, policia] ;

Oper = baixo – [ladrao, policia] ; no

2.3 b) Defina o predicado **invalida(+Margem1, +Margem2)** que dadas as listas de elementos que estão em cada uma das margens, suceda se a situação (estado) for inválida ou falhe caso contrário (ou seja suceda se não respeitar as restrições). (1 Val.).

Exemplos:

?- invalida([barco, filha, filho, ladrao, policia], [filha, filho, mae, pai]).

no

?- invalida([barco, filha, filha, filho, mae, ladrao, policia], [filho, pai]). % Mãe sozinha com filho na Margem1!

yes

?- invalida([barco, filha, filho, ladrao], [filha, filho, mae, pai, policia]). % Ladrão sozinho com elementos da família

yes

2.3 c) Suponha que dispõe dos predicados **inserirOrdenado(+Elem, +ListaIni, -ListaRes)** que adiciona o elemento **Elem** a uma lista **ListaIni** ordenada alfabeticamente mantendo a ordenação na lista resultado (**ListaRes**) e **retirar(+Elem, +ListaIni, -ListaRes)** que retira, caso exista, a primeira ocorrência de **Elem** da lista **ListaIni** gerando a lista **ListaRes**. Defina o predicado **executar(+Margem1, +Margem2, +Oper, -NovaMargem1, -NovaMargem2)**, que execute um dado operador, ou seja que mova os elementos de uma margem para a outra. **(1 Val.)**

Exemplo:

?- executar([barco, filha, filho, ladrao, policia], [filha, filho, mae, pai], baixo - [ladrao, policia], M1, M2).

M1 = [filha, filho], M2 = [barco, filha, filho, ladrao, mae, pai, policia].

2.3 d) Construa o predicado **resolver(-ListaOperadores)**, que retorne a lista de operadores que é necessário aplicar para resolver o problema. Tenha cuidado para evitar estados repetidos e estados inválidos. **(1.5 Val.)**

Exemplo:

?- resolver(Lista).

Lista = [baixo - [policia, ladrao], cima - [policia], ... % Pense na solução...

GRUPO III – Programação em Lógica com Restrições (6.5 val)

3.1) A multiplicação representada ao lado deve dar um resultado correcto utilizando os seguintes oito dígitos: 2, 2, 3, 3, 4, 5, 7, 7. Construa um programa **puzzleNum (-Lista)** eficiente capaz de resolver o problema utilizando PLR.

Sugestão: Utilize a restrição global_cardinality **(2 Val.)**

Exemplo:

?- puzzleNum(Lista).

Lista = [5, 3, 2, 7, 3, 7, 2, 4].

$$\begin{array}{r}
 \begin{array}{ccc}
 \boxed{5} & \boxed{3} & \boxed{2} \\
 & \times & \boxed{7} \\
 \hline
 \boxed{3} & \boxed{7} & \boxed{2} & \boxed{4}
 \end{array}
 \end{array}$$

3.2) Quatro estudantes da FEUP: Artur, Bárbara, Carlos e Diana, partilham um apartamento. Quatro jornais são entregues todos os dias no apartamento: Jornal de Notícias, O Jogo, Público e A Bola. Cada um dos estudantes gosta de ler “sossegadamente” todos os jornais durante um dado intervalo de tempo cada um (ver abaixo). Os quatros estudantes vão juntos de carro para a FEUP onde no máximo têm de estar à hora de almoço (13h00m). Dado que o Artur se levanta às 8h30m, a Bárbara e o Carlos às 8h45m e a Diana às 9h30m, qual é a hora mínima em que podem sair todos juntos de casa para a FEUP (tendo, claro, lido calmamente os seus jornais)? Pretende-se formular o problema utilizando o paradigma da Programação em Lógica com Restrições (Domínios Finitos).

Artur	Bárbara	Carlos	Diana
JN 60 min	A Bola 5 min	O Jogo 75 min	Público 90 min
O Jogo 30 min	O Jogo 15 min	A Bola 3 min	JN 2 min
A Bola 2 min	JN 10 min	JN 25 min	O Jogo 1 min
Público 6 min	Público 30 min	Público 10 min	A Bola 1 min

3.2 a) Quais as variáveis e respectivos domínios para este problema. **(1 Val.)**.

3.2 b) Quais as restrições rígidas/flexíveis e qual a função de avaliação da solução do problema? **(1 Val.)**.

3.2 c) Construa um programa PLR **estudantes(-Solucao)**, que seja capaz de resolver este problema e problemas semelhantes. Suponha que os dados do problema são representados da forma indicada. **(2.5 Val.)**.

jornais(4, [jn, ojogo, publico, abola]).

peessoas(4, [artur, barbara, carlos, diana]).

tempos(artur, [jn-60, ojogo-30, abola-2, publico-6]).

tempos(barbara, [jn-10, ojogo-15, abola-5, publico-30]).

tempos(carlos, [jn-25, ojogo-75, abola-3, publico-10]).

tempos(diana, [jn-2, ojogo-1, abola-1, publico-90]).

horas_acordam([30, 45, 45, 90]).

Nota: Como simplificação utiliza-se a unidade minutos (desde as 8.00) e efectuam-se todas as contas nesta unidade, ou seja: horas_acordam([30, 45, 45, 90]) significa que o Artur se levanta às 8.30, a Barbara e o Carlos às 8.45 e a Diana às 9.30 (90 minutos depois das 8.00).